

Contents

Chapter 15: How Coding Agents Work	1
1. The Coding Agent as a Control System	2
2. The Agent Harness	4
3. Context Management	5
4. Tool Use	6
5. Planning	8
6. Execution	9
7. Verification	10
8. Feedback and Iteration	12
9. Compaction	13
10. Subagents	14
11. Permissions	15
12. The Repository as the Agent’s Environment	16
13. Why Tests Become Part of the Agent’s Reasoning System	17
14. Coding Agents as Search Systems	18
15. The Full Architecture	19
16. What Actually Determines Coding-Agent Quality?	20
17. Exercise — Build a Minimal Coding Agent	22
18. Key Takeaways	23

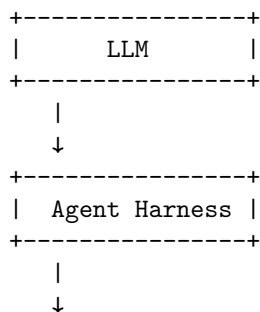
Chapter 15: How Coding Agents Work

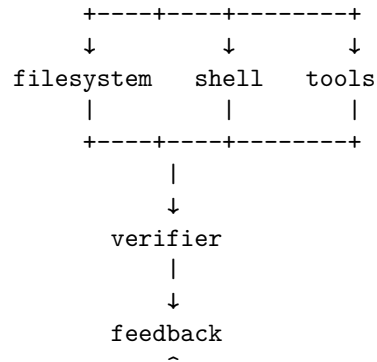
Coding agents are among the clearest examples of the transition from **LLM-as-a-component** to **LLM-as-a-system controller**.

A conventional code-generation workflow looks roughly like:

```
Prompt
↓
LLM
↓
Code
```

A coding agent looks fundamentally different:





The important shift is that the model does not simply produce the final artifact. It participates in a **closed-loop engineering process**:

Observe → reason → act → verify → update context → act again.

This architecture is the foundation behind modern coding agents such as IDE agents, terminal-based agents, repository assistants, and autonomous software-engineering systems.

The central engineering question is therefore no longer:

How good is the model at writing code?

It is:

How effectively can a system use a probabilistic model to perform a reliable software-engineering process?

1. The Coding Agent as a Control System

A useful abstraction is to model a coding agent as a feedback controller.

Let:

$$S_t$$

represent the state of the software environment at time t . This includes:

- source files
- configuration
- dependencies
- build artifacts
- test results
- git state
- environment variables
- tool outputs

- repository conventions

The agent observes some representation of that state:

$$O_t = \mathcal{O}(S_t)$$

and uses its context and reasoning to select an action:

$$A_t \sim \pi_\theta(A \mid C_t, O_t)$$

where:

- C_t is the agent's current context
- π_θ is the LLM-driven policy
- A_t might be a file edit, shell command, test invocation, search operation, or tool call

The environment then transitions:

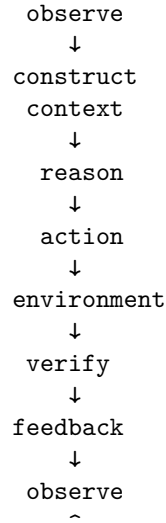
$$S_{t+1} = T(S_t, A_t)$$

The agent receives new observations:

$$O_{t+1} = \mathcal{O}(S_{t+1})$$

and continues.

Thus:



This is much closer to **closed-loop control** than to traditional code generation.

The LLM supplies the policy. The harness supplies the control machinery. The repository is the environment. Tests and other verification mechanisms provide feedback.

That distinction explains why coding-agent performance depends on much more than the underlying model.

2. The Agent Harness

The **agent harness** is the software surrounding the LLM.

This distinction is critical.

An LLM API generally provides something like:

`messages → model → response`

A coding agent needs to provide:

```
model
  ↓
interpret response
  ↓
identify requested action
  ↓
authorize action
  ↓
execute tool
  ↓
capture result
  ↓
update state
  ↓
construct new context
  ↓
invoke model again
```

The harness is therefore the **runtime environment for the model's reasoning process**.

It typically implements:

- tool definitions
- tool execution
- state management
- context construction
- permissions
- retry logic
- error handling
- output parsing
- execution limits
- context compaction

- subagent orchestration
- verification
- stopping conditions
- logging and observability

A useful mental model is:

The LLM is not the coding agent. The LLM is a reasoning component inside the coding agent.

This is analogous to a CPU inside an operating system. The CPU performs computation, but the operating system determines how computation interacts with memory, processes, devices, permissions, and the external environment.

3. Context Management

The first major challenge is **context**.

A repository can contain millions of tokens of potentially relevant information:

```
repository
+-- source/
+-- tests/
+-- documentation/
+-- configuration/
+-- dependencies/
+-- generated files/
+-- build artifacts/
`-- git history
```

The model cannot simply receive the entire repository on every iteration.

The harness must construct an appropriate context:

$$C_t = C_{\text{system}} + C_{\text{task}} + C_{\text{repository}} + C_{\text{history}} + C_{\text{tools}} + C_{\text{feedback}}$$

The problem is not merely fitting within the context window.

It is **selecting the information that matters**.

Too little context causes errors:

```
missing API contract
↓
incorrect implementation
```

Too much irrelevant context can also cause errors:

```
context pollution
↓
reduced signal-to-noise ratio
```

↓
weaker reasoning

A strong coding agent therefore performs something resembling **context engineering**.

It may:

1. inspect the repository structure
2. search for relevant symbols
3. locate tests
4. inspect configuration
5. read only relevant files
6. inspect call sites
7. examine recent tool results
8. summarize older interactions
9. discard irrelevant information

The effective context is therefore dynamically constructed rather than statically supplied.

4. Tool Use

A coding agent needs access to the external world.

Typical tools include:

Filesystem

```
+-- read file
+-- write file
+-- edit file
`-- search
```

Shell

```
+-- run tests
+-- build
+-- install dependencies
+-- git
`-- execute programs
```

Repository tools

```
+-- symbol search
+-- diff
+-- history
`-- code navigation
```

External tools

```
+-- documentation
+-- issue trackers
+-- package registries
`-- APIs
```

The model does not directly manipulate the filesystem or execute commands.

Instead, it emits a structured request:

```
tool = run_tests
arguments = {...}
```

The harness executes it and returns the result.

This creates an important separation:

```
LLM
|
| proposes
↓
tool invocation
|
| executes
↓
external environment
|
| returns observation
↓
LLM
```

The model therefore operates through an **action interface**.

This interface is one of the most important pieces of agent architecture.

Poor tools make the agent ineffective even if the model is highly capable.

For example, giving an agent only:

```
read_file()
write_file()
```

forces it to reconstruct capabilities that could have been provided directly through:

```
search_code()
find_symbol()
run_tests()
inspect_git_diff()
run_typechecker()
```

Tool design is consequently an important form of **agent engineering**.

5. Planning

A coding task often requires multiple dependent actions.

Consider:

Add OAuth authentication to the application.

The task might decompose into:

1. Inspect current authentication architecture
2. Identify framework and dependencies
3. Inspect user model
4. Inspect existing session handling
5. Design OAuth integration
6. Modify configuration
7. Implement provider integration
8. Update routes
9. Add tests
10. Run tests
11. Fix failures
12. Review diff

A coding agent therefore needs some form of planning.

Planning may be:

Explicit The agent constructs a visible plan:

Plan:

1. Inspect auth module
2. Add OAuth dependency
3. Implement callback
4. Add tests
5. Run test suite

Implicit The model simply reasons about the next best action at each step.

Hierarchical A high-level task is decomposed into subtasks:

Feature

```
+-- backend
|   +-- API
|   `-- database
+-- frontend
|   +-- UI
|   `-- state
`-- tests
    +-- unit
    `-- integration
```


Modern systems can also delegate parts of the problem to **subagents**.

Planning introduces an important distinction:

A coding agent is not necessarily executing a predetermined plan. It is often continuously replanning as new information arrives.

This matters because software environments are only partially known before execution.

6. Execution

Execution converts reasoning into environmental changes.

A typical trajectory might look like:

LLM:

"Inspect src/auth.py"

↓

`filesystem.read(src/auth.py)`

↓

result returned

↓

LLM:

"The authentication state is stored in SessionManager."

↓

`filesystem.read(src/session.py)`

↓

LLM:

"Modify SessionManager."

↓

`filesystem.edit(...)`

↓

```
shell.run("pytest")
```

↓

```
FAILED: 3 tests
```

↓

```
LLM:  
"Inspect failures."
```

↓

```
...
```

Notice what is happening.

The model is not generating one large answer.

It is repeatedly alternating between:

```
reasoning
```

↓

```
action
```

↓

```
observation
```

This is sometimes called an **agent trajectory**.

A trajectory can be represented as:

$$\tau = (O_0, A_0, O_1, A_1, \dots, O_n)$$

The quality of the final result depends not only on the quality of individual actions, but on the quality of the entire trajectory.

7. Verification

Verification is arguably the most important architectural component.

An agent can generate plausible code that is completely wrong.

Therefore:

Generation without verification is not software engineering.

Verification mechanisms include:

Deterministic checks

- pytest
- mypy
- ruff
- eslint
- tsc
- cargo check
- go test

Build verification

- compile
- package
- link
- container build
- deployment validation

Behavioral verification

- unit tests
- integration tests
- end-to-end tests

Static analysis

- type checking
- linting
- security scanning
- dependency analysis

Repository-specific checks

- API compatibility
- schema validation
- migration checks
- golden tests

Human verification For high-risk changes, a human remains part of the loop.

The key idea is that verification converts vague model uncertainty into concrete evidence.

Instead of:

"I think this works."

the system obtains:

```
pytest:  
42 passed
```

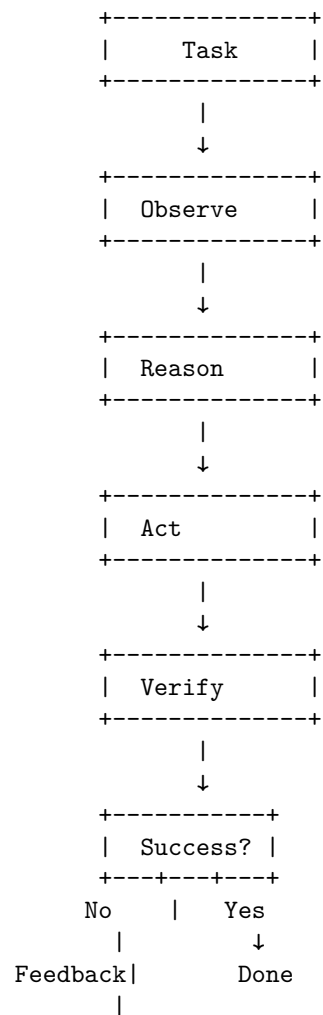
or:

```
pytest:  
3 failed
```

That feedback can then become input to the next reasoning step.

8. Feedback and Iteration

The basic coding-agent loop is therefore:



+-----> Reason

This loop is powerful because the agent does not have to be correct initially.

Suppose the model writes incorrect code with probability p .

If verification reliably detects the error and the agent can repair it, the system can recover from many initial mistakes.

This changes the engineering objective.

We no longer need:

$$P(\text{correct first attempt}) \approx 1$$

Instead, we want:

$$P(\text{eventual success} \mid \text{feedback} + \text{iteration})$$

to be high.

This is a profound shift.

The system's capability emerges partly from **error detection and recovery**, not merely from initial generation quality.

9. Compaction

Agent trajectories can become extremely long.

Imagine:

Read 30 files

↓

Run 15 searches

↓

Edit 12 files

↓

Run 8 test commands

↓

Fix 5 failures

↓

Inspect git history

↓

Run integration tests

Sending the complete history back to the model indefinitely is inefficient and eventually impossible.

The harness therefore needs **context compaction**.

Conceptually:

$$H_{0:t} \rightarrow \text{Compress}(H_{0:t}) \rightarrow S_t$$

where $H_{0:t}$ is the complete interaction history and S_t is a compact representation of the state that matters.

A useful summary might preserve:

Task:

Implement OAuth authentication.

Completed:

- Added OAuth dependency
- Modified SessionManager
- Added callback route

Current state:

- Unit tests pass
- Integration test fails

Known issue:

- Callback does not preserve redirect URL

Relevant files:

- src/auth.py
- src/session.py
- tests/test_oauth.py

The challenge is that **compression is lossy**.

Discard the wrong information and the agent may later repeat work or make an incorrect assumption.

Compaction is therefore not merely a token optimization.

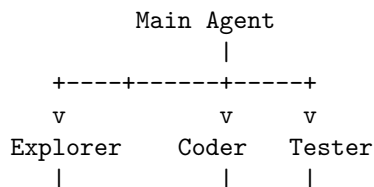
It is a form of **state management**.

10. Subagents

Complex coding tasks can be decomposed among specialized agents.

For example:

```
```text
```



$\begin{matrix} & v & & v & & v \\ & analysis & & edits & & valid \end{matrix}$

A subagent might specialize in:

- repository exploration
- debugging
- test generation
- documentation
- security analysis
- code review
- architecture analysis

The main agent can delegate:

"Find where authentication state is managed  
and report the relevant files and invariants."

The explorer returns a concise result.

This can reduce context pressure on the main agent.

However, delegation introduces additional engineering problems:

- task decomposition
- subagent context construction
- result synthesis
- consistency
- duplicate work
- synchronization
- permission boundaries
- cost management

Subagents therefore do not automatically make an agent better.

They are useful when the problem has enough **structural separability** to justify parallel or hierarchical reasoning.

---

## 11. Permissions

Coding agents can potentially perform dangerous operations.

A shell-enabled agent may be able to:

delete files  
modify repositories  
install packages  
access credentials  
send network requests  
change infrastructure

Therefore, tool access must be governed by permissions.

A useful abstraction is:

$$\text{AllowedActions} = f(\text{user}, \text{task}, \text{environment}, \text{risk})$$

For example:

Read files	yes
Edit source	yes
Run tests	yes
Git diff	yes
Git commit	?
Network access	?
Package installation	?
Delete repository files	?
Production deployment	no

Permissions can be:

**Static** A fixed sandbox determines what the agent can do.

**Dynamic** The agent requests authorization for sensitive operations.

Agent:

"I need to execute this command because..."

User:

Approve / Reject

**Policy-based** The harness evaluates the action against predefined rules.

The important principle is:

**The model should not be the ultimate authority over its own permissions.**

The harness must enforce the boundary independently of the model's intentions.

---

## 12. The Repository as the Agent's Environment

A useful way to understand coding agents is to treat the repository as an **external environment**.

The model has only partial observability.

It does not initially know:

- every file



- every dependency
- every invariant
- every runtime behavior
- every undocumented convention
- every deployment constraint

It discovers these through actions.

This resembles a partially observable decision process:

Hidden repository state  $\rightarrow$  observations  $\rightarrow$  actions  $\rightarrow$  new observations

This explains why repository exploration is not overhead.

**Exploration is part of reasoning.**

A strong agent may spend substantial time inspecting the environment before changing anything.

That is often rational.

---

### 13. Why Tests Become Part of the Agent's Reasoning System

In conventional development, tests are often treated as a final validation mechanism.

In an agentic system, tests become an **information source**.

Consider:

```

Agent modifies code
 ↓
pytest
 ↓
3 failures
 ↓
failure messages
 ↓
agent updates hypothesis
 ↓
new modification

```

The test suite is now part of the agent's cognitive loop.

This leads to a broader principle:

**A good verifier is not merely a gate. It is a source of information.**

Compare:

BUILD FAILED

with:

test\_auth\_redirect\_preserves\_original\_url FAILED

Expected:

"/dashboard"

Received:

"/"

Stack:

...

The second provides substantially more information for the next reasoning step.

Consequently, engineering the verification system can improve agent performance even without changing the model.

---

## 14. Coding Agents as Search Systems

Another useful perspective is to view coding as a search problem.

Suppose the repository state is  $S_0$ .

The agent explores a sequence:

$$S_0 \xrightarrow{A_0} S_1 \xrightarrow{A_1} S_2 \rightarrow \dots \rightarrow S_n$$

The objective is to find a state satisfying a set of constraints:

$$S_n \models \text{requirements, tests, security, architecture, style}$$

The agent is effectively searching through a large space of possible modifications.

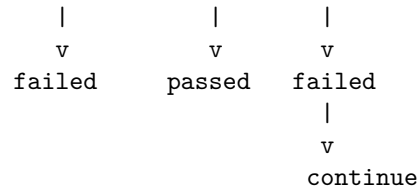
Verification prunes that search space.

For example:

```text

```

      Candidate changes
          |
    +----+----+----+
    v      v      v
Change A  Change B  Change C
    |      |      |
    v      v      v
  tests    tests    tests
```

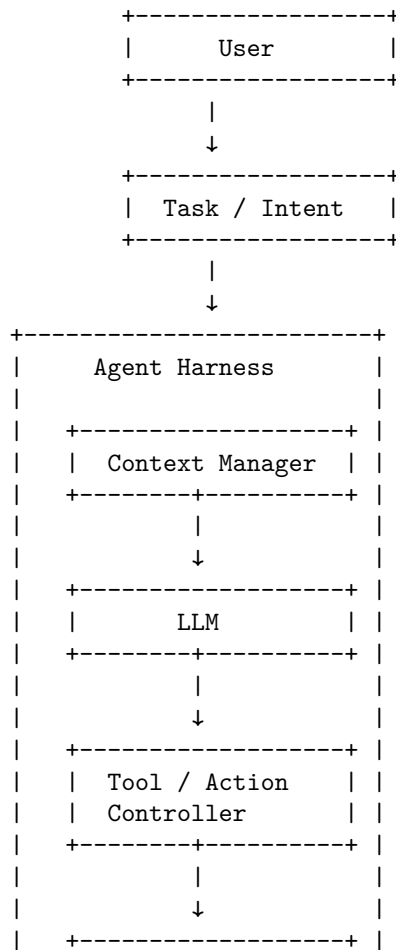


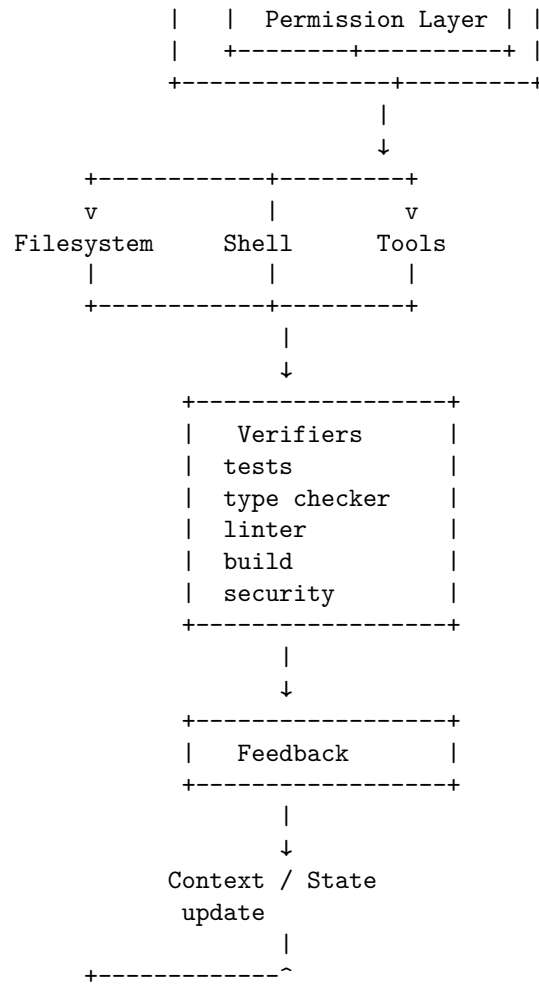
This makes verification analogous to a search heuristic.

The stronger the verifier, the more efficiently the system can eliminate incorrect trajectories.

15. The Full Architecture

Putting everything together gives a more realistic architecture:





The architecture reveals that a coding agent is really a composition of several engineering systems:

| |
|---|
| Coding Agent = Model + Harness + Tools + Context + Verification + Control |
|---|

The model is essential, but it is only one component.

16. What Actually Determines Coding-Agent Quality?

It is tempting to rank coding agents by model benchmark scores alone.

That is inadequate.

A more complete model is:

$$Q_{\text{agent}} = f(Q_{\text{model}}, Q_{\text{context}}, Q_{\text{tools}}, Q_{\text{planning}}, Q_{\text{verification}}, Q_{\text{recovery}}, Q_{\text{permissions}})$$

Consider two systems using the same model.

System A

```
LLM
↓
generate patch
↓
return
```

System B

```
LLM
↓
inspect repository
↓
search symbols
↓
construct context
↓
edit
↓
run tests
↓
analyze failures
↓
repair
↓
rerun
↓
typecheck
↓
review diff
↓
return
```

The underlying model is identical.

Yet System B can be dramatically more capable because it provides a better **engineering loop**.

This is one of the most important lessons of agent engineering:

System architecture can amplify model capability.

17. Exercise — Build a Minimal Coding Agent

For this day's project, build a small coding agent around a repository.

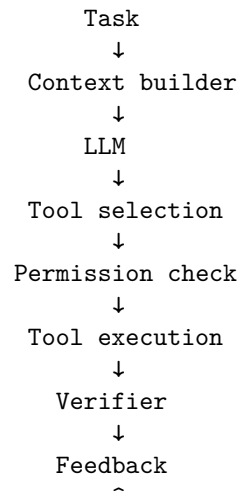
The objective is not to build a production-grade autonomous programmer.

The objective is to understand the control loop.

Your agent should support:

1. Receive a coding task
2. Inspect repository files
3. Search the codebase
4. Read relevant files
5. Propose an implementation
6. Modify files
7. Run tests
8. Read failures
9. Iterate
10. Stop when verification succeeds

A minimal architecture might be:



Instrument the system.

Record:

iteration number
tool calls
tokens consumed
files read
files modified
tests executed

test results
errors
time per iteration
final outcome

Then intentionally introduce failures.

For example:

Task:
Add a function that parses configuration.

Experiment:
Inject an incorrect implementation.

Observe:
Can the agent detect the failure?
Can it diagnose the cause?
Can it repair the implementation?
How many iterations does it require?

The goal is to move from:

“The model generated code.”

to:

“The system successfully navigated a software environment to a verified state.”

That is the fundamental conceptual transition.

18. Key Takeaways

1. **A coding agent is not an LLM.** It is an LLM embedded inside a runtime that provides context, tools, state, permissions, execution, and verification.
2. **The agent is a closed-loop system.** Its fundamental cycle is:
Observe → Reason → Act → Verify → Feedback → Repeat
3. **The harness is a first-class engineering component.** Tool execution, context management, retries, permissions, compaction, and stopping conditions strongly influence system capability.
4. **Context engineering is central.** The agent must dynamically determine what repository information is relevant rather than blindly passing the entire codebase to the model.

5. **Tools extend the model into the environment.** Filesystem, shell, repository, documentation, and verification tools turn an LLM from a text generator into an interactive engineering system.
6. **Verification closes the loop.** Tests, type checkers, linters, builds, and other verifiers transform uncertain model output into measurable feedback.
7. **Iteration changes the capability equation.** The important metric is not only whether the agent is correct on its first attempt, but whether it can detect and recover from errors.
8. **Compaction is state management, not merely token optimization.** Long-running agents need to preserve important state while discarding irrelevant trajectory history.
9. **Subagents provide hierarchical decomposition.** They can specialize exploration, implementation, testing, review, or other tasks, but introduce orchestration and consistency costs.
10. **Permissions must be enforced outside the model.** An agent should not be trusted to determine the boundaries of its own authority.
11. **Tests become part of the agent’s reasoning environment.** A well-designed verifier is not merely a final gate; it provides information that guides subsequent actions.
12. **Coding agents are search-and-control systems.** They explore a space of possible repository states, using verification to eliminate incorrect trajectories.
13. **The future unit of software engineering is increasingly the trajectory, not the generated code fragment.** The important question becomes:

Can the system reliably move from an underspecified software state to a verified desired state?

That is the core idea behind modern coding agents.